Copyright 2026
No Unauthorized Duplication Permitted

Linux Commands for the Control of McMaster AEV

In this lab, you will practice the essential Linux commands that you might need to program and control the McMaster Autonomous Electric Vehicle throughout this course. We expect you to read this lab manual carefully and study the required material. **This lab must be completed individually, not in groups.** Please **avoid copying and pasting the commands** given in this lab manual. When you copy and paste commands, syntax errors might occur. We strongly recommend you to type the codes manually instead of copying and pasting them from this document or from other sources available on the internet.

In this lab, you will focus on the fundamentals of Linux commands and their meanings, and you will practice how to use them. This will help you understand the project that you will be completing in this course. Please follow the following safety procedures for this lab.

Important Safety Procedures to Follow

- In this lab, you should not work with the McMaster Autonomous Electric Vehicle (AEV).
- You can use a computer to complete the objectives of this lab.
- You should not power up McMaster AEV for this experiment.
- You should not connect the battery to McMaster AEV for this experiment.

Objective 1: Introduction to Linux and Your Linux Cheat Sheet

Linux has been around since the mid 1990s and has since reached a user base that spans the globe. Linux is everywhere. It is in your phones, your thermostats, refrigerators, televisions, and your cars. You will also use Linux to program and control McMaster AEV and its drive system.

Linux is a reliable and secure operating system. Just like Windows, iOS, and MacOS, Linux is an operating system. An operating system is software that manages the hardware resources associated with your desktop or laptop. Simply, an operating system manages the communication between your software and hardware.

Copyright 2026
No Unauthorized Duplication Permitted

The Linux operating system comprises several different components:

Bootloader, is software that manages the boot process of your computer. For most users, this is simply a splash screen that pops up and eventually goes away to boot into the operating system.

Kernel, is the core of the system and manages the Central Processing Unit (CPU), memory, and peripheral devices. The Kernel is the lowest level of an operating system.

Daemons, are background services (printing, sound, scheduling, etc.) that either start up during the boot or after you log onto the desktop.

Init System, is a subsystem that bootstraps the user space and is charged with controlling daemons. One of the most widely used Init Systems is **systemd**. It is the Init System that manages the boot process, once the initial booting is handed over from the bootloader.

Desktop Environment, is the component that the users interact with. There are many desktop environments to choose from, such as GNOME, Cinnamon, Mate, Pantheon, Enlightenment, KDE, Xfce). Each Desktop Environment includes built-in applications, such as file managers, configuration tools, web browsers, and games. Desktop Environments do not offer the full array of **Applications**. For example, Ubuntu Linux has the Ubuntu Software Center which allows you to search among thousands of Applications and install them from one centralized location.

On **Avenue to Learn folder for this lab**, you will find videos on Linux commands. **You must watch the videos and prepare a cheat sheet for your reference for the Linux commands you learned.** You can prepare your cheat sheets using your tablet or a paper, but it **must be handwritten**. **You are expected to complete Objective 1 before your lab session.**

Objective 2: Practice of the Linux Commands

This objective will help you use and run some Linux commands. It will allow you to practice the knowledge you gained in Objective 1. **You are expected to complete Objective 2 before your lab session.**

Copyright 2026
No Unauthorized Duplication Permitted

You can use an Online Linux Terminal to practice the Linux commands. Table 1 shows a list of shortcut keys that can come handy while you practice using the Linux commands.

Table 1. Shortcut keys

Shortcut Key	Function
`Ctrl + d*`	Keyboard input ends or exits the terminal
`Ctrl + s*`	Pause the current program and press any key to resume it
`Ctrl + z`	Put the current program into the background
`Ctrl + a`	Move the cursor to the beginning of the input line, equivalent to Home
`Ctrl + e`	Move the cursor to the end of the input line, equivalent to End
`Ctrl + k`	Remove content from the place the cursor is currently at to the end of the line
`Alt + Backspace`	Delete a word forward
`Shift + PgUp`	Scroll up the terminal
`Shift + PgDn`	Scroll down the terminal

*For this lab, while using the online portal, you will only need to use `ctrl+d` and `ctrl+c`. The rest of the shortcut keys might come handy in future labs.

Hint: If the cursor on the Linux command prompt is flashing, but you cannot enter a command, or if you need to immediately stop the execution and restore to your controllable state, you can use `Ctrl+c`. It forces the program to terminate, but it will not exit the terminal. However, please do not use `Ctrl+c` in every situation. In some cases, when the Linux command prompt is not responding, it might be possible that it takes time the program to start or finish. Therefore, do not rush to use `Ctrl+c`.

In this lab manual, the commands to enter at the command prompt are given in the following format:

```
unix> This is the command that you should enter
```

The `unix>` part **should not be typed,** and it might not look the same on your computer. It might look like `username@computername:/current-directory$`, which is a combination of your username, computer name, and the current directory you are in. Thus, it changes while you work. For simplicity, we use `unix>` in the lab manual.

Now we begin practicing the Linux commands. In Objective 3, you will be **asked to complete an exercise** on how to use the Linux commands and submit your results.

Copyright 2026
No Unauthorized Duplication Permitted

1. `pwd`: Show the current directory

Pathnames enable you to work out where you are in relation to the whole file system. If you apply this command when you start the Linux terminal the first time, it will probably give you the path of your root directory.

```
unix> pwd
```

2. `ls`: List files

As you open the online Linux terminal, your current working directory is your home directory. Your home directory has the same name as your username, such as your *macID*. Your home directory is where your personal files and subdirectories are saved. In order to find what is in your home directory or any directory you are currently working in, use the `ls` command. `ls` is short for list. There may be no files in your home directory, in which case, nothing will print but the command prompt again.

```
unix> ls
```

The `ls` command does not show the hidden files by default. Hidden files have file names that begin with a dot (.). To list all the files in your home directory, including the hidden files, use the `-a` option with the `ls` command.

```
unix>  ls -a
```

3. `mkdir`: Make directory

In order to make a directory called `macID` inside your current working directory, type:

```
unix>  mkdir macID
```

Use the `ls` command to see the directory you have just created:

```
unix>  ls
```

Copyright 2026
No Unauthorized Duplication Permitted

4. `cd`: Change to a different directory

The command `cd newDirectory` changes the working directory to `newDirectory`. You can think of the working directory as the directory you are in, i.e. your current position in the file system. In order to change the working directory to the `macID` directory you just created, you can type:

```
unix>  cd macID
```

Now, you can check your working directory:

```
unix>  pwd
```

The pathname should look something like `/root/macID`. This means that the directory titled with you `macID` is inside the `root` directory, which is the directory that contains all user accounts. In order to go up one level in the directory tree, you can type:

```
unix>  cd ..
```

Please note that there should be a space between the command `cd` and the dots. At this point, you should be back in your root directory.

Now go back to your `macID` directory once again:

```
unix>  cd macID
```

Make a new directory in there named `lab1`:

```
unix>  mkdir lab1
```

In order to enter the new directory, you just created type:

```
unix>  cd lab1
```

In order to jump back to the `root` directory regardless of how deep in the file tree you happen to be, use either of the following commands:

```
unix>  cd
```

or

```
unix>  cd ~
```

Copyright 2026
No Unauthorized Duplication Permitted

5. `cp`: Copy file

In order to practice `cp` command, first enter the `lab1` directory, and create two empty files there using the `touch` command which can be used to create new files, similar to `nano`.

```
unix>  cd macID/lab1
```

```
unix>  touch file1
```

```
unix>  touch file2
```

`cp` command needs two arguments. The first one is the name of the file to be copied. The second one is the name of the copied file. In order to make a copy of the file in the current directory, use:

```
unix>  cp file1 file2
```

This copies `file1` as `file2`. If `file2` does not already exists, then the `cp` command creates it. If it does exist, then the `cp` command replaces it with a copy of `file1`. For example, try:

```
unix>  cp file1 file4
```

Then check the content of the directory by:

```
unix>  ls
```

You will notice that you now have a new file called `file4`. In order to copy a file from one directory to another, type the following:

```
unix>  cp file1 /root/macID
```

This copies `file1` from the `Lab1` directory to the `macID` directory. Then, go to the `macID` directory:

```
unix>  cd ..
```

and use the `ls` command to check that `file1` has been copied to the `macID` directory:

```
unix>  ls
```

You should now have the new file you just copied `file1`.

Copyright 2026
No Unauthorized Duplication Permitted

6. `mv`: Move/rename a file

We can use the `mv` command to rename `file4` as `document1`. First, navigate to the `lab1` directory:

```
unix>  cd lab1
```

```
unix>  mv file4 document1
```

Then, type `ls` to confirm that the file name has changed:

```
unix>  ls
```

In order to move `document1` to your `macID` directory, apply the following command:

```
unix> mv document1 ~/macID
```

Navigate to your `macID` directory:

```
unix>  cd ..
```

Then, type `ls` to confirm that `document1` has been moved to your `macID` directory.

```
unix>  ls
```

7. `rm`: Delete file

In order to delete (remove) a file, you can use the `rm` command:

```
unix>  rm ~/macID/lab1/file2
```

Caution! The `rm` command is powerful. Once you remove a file, recovering that file can be exceedingly difficult, if not impossible. The command line does not provide a check before you delete a file. In order to avoid deleting a file accidentally, you can use the following:

```
unix>  rm -i ~/macID/lab1/file2
```

You should get a message that looks like:

```
rm: remove regular empty file '/root/macID/lab1/file2'?
```

Then type `Y` meaning Yes or `N` meaning No

Then type `ls` to confirm that `file2` has been deleted after you navigate to the `lab1` directory:

```
unix>  ~/macID/lab1
```

```
unix>  ls
```

Copyright 2026
No Unauthorized Duplication Permitted

8. `rmdir`: Delete (remove) directory:

When you are in the `macID` directory, if you apply the following command to remove `lab1` directory:

```
unix>  rmdir lab1
```

You will get an error, because `lab1` directory is not empty. You can use the following command to remove the `lab1` directory:

```
unix>  rm -r ~/macID/lab1
```

`-r` means recursively. Therefore, the above command can be used to empty the files inside the directory and then delete the directory itself. Then, type `ls` to confirm that the directory lab1 has been removed:

```
unix>  cd ~/macID
```

```
unix>  ls
```

9. `nano`: creating a text file

In your macID directory, create a text file called `file_nano`

```
unix>  nano file_nano.txt
```

When the text file opens up, type `Hello World!`
Save the file, using `Ctrl+O` and then press `enter`
Now exit the file, using `Ctrl+X`

To view the text file, type

```
unix>  cat file_nano.txt
```

In order to clear the terminal window, you can use the `clear` command:

```
unix>  clear
```

10. Working with Packages:

You will be working with packages in the future labs to control the McMaster AEV. This section will provide the basic commands for working with packages. This section is provided to get you familiar

Copyright 2026
No Unauthorized Duplication Permitted

with working with packages for future labs. It is highly recommended that you include these commands in your cheat sheet.

sudo:

The `sudo` command is an abbreviation for *super user do*. It is a Linux command that allows programs to be executed as a super user – known as root user. Here is an example for a typical use of the `sudo` command:

```
sudo apt install curl
```

Here, `sudo` was used before a restricted command where the system will ask you for your password. After inputting your password, you can run restricted commands for about 15 mins without inputting your password. After that period is over, you will have to input your password again.

gedit:

`gedit` is a text editor just like nano and it is the default Graphical User Interface (GUI) text editor in Ubuntu operating system. It supports most standard text editor features as well as many advanced features. For example, look at the following command should open up the *.bashrc file* which is a text file that you can edit:

```
sudo gedit ~/.bashrc
```

apt:

`apt` is known as *Advanced Package Tool*. It is a collection of tools used to install, update, remove, and otherwise manage software packages on Ubuntu. For example, in the following command, we need `apt` to install `xfce4`, which is a software package.

```
sudo apt install xfce4
```

wget:

The name of the `wget` command comes from *Word Wide Web* and *get*. It is used to download files from the internet. It supports HTTP, HTTPS, FTP, and FTPS internet protocols. For Example, look at the following command:

```
wget https://www.nomachine.com/free/arm/v8/deb -O nomachine.deb
```

Copyright 2026
No Unauthorized Duplication Permitted

Here, the command `wget` downloads the NoMachine DEB package.

`dpkg`:

`dpkg`, short for *Debian Package Manager*, is the default package manager on Ubuntu. You use `dpkg` to install, configure, upgrade, or remove Debian packages, and retrieve information of these Debian packages. For example, in the following command `dpkg` is used to install the NoMachine software. You will use the NoMachine software in the future labs.

```
sudo dpkg -i nomachine.deb
```

Objective 3: Exercise on Linux Commands

Now that you have gained practice on using Linux commands, you can try the following exercise on your own. You should **finish this exercise in one sitting so you do not lose your work**. Please follow the tasks as described and take screenshots where it states, before you continue with another task. Place your screenshots in a WORD document and label each screenshot with which step it represents. **Submit your file to Avenue to Learn as a PDF file.** Your submission file should have the following sections:

- Steps 1 – 4
- Step 7
- Pyhton codes: addition.py and MyInfo.py
- Step 10
- Steps 12 – 16
- Steps 18 – 21
- Steps 23 – 26
- Steps 28 – 31: List the number of each word in Step 31.

Your submission will be marked for the correct use of the commands. We will also be looking for proof of your work. If you created a directory, you need to confirm that it was created.

Copyright 2026
No Unauthorized Duplication Permitted

Here are the tasks you need to accomplish in this exercise.

1. Make a new directory called `home_yourMacID` in the root director, e.g. `home_bilginb`
2. Verify that step (1) was successful.
3. Check what hidden files are present in your root directory
4. Make three directories inside your `home_yourMacID` directory. Name them: `yourMacID_folderA`, `yourMacID_folderB`, and `yourMacID_backup`
5. Take a screenshot of your terminal and save it to your computer as Steps 1-4.

6. Create four text files in the `home` directory. Name them the following and make sure they contain the following information in the same order. In each file, after you enter an item, press enter and add the next item under the previous one. Do not include the semicolons (;) in the files.

   `MyCalendar.txt`: March; August; July; February; October; May; September; June; April

   `MyFamily.txt`: Brandon; Sandra; Peter; Jennifer; Graham

   `MyDailySteps.txt`: 5000; 1800; 17000; 560; 900

   `MyToDoList.txt`: I make my bed in the morning; I eat pancakes for breakfast; I finish my homework in the evening; I go to dinner later in the evening

7. View the text files you just created to make sure you typed everything correctly
8. Take a screenshot of your terminal and save it to your computer as Step 7.

9. Create two python files in the directory `yourMacID_backup`. In order to find out the python version in the online terminal, type `python` command in the online Linux terminal. Take a screenshot of your codes for both files.

   a. Create the python program that adds two numbers with user input and name it `calculation.py`.

   b. Create a python program that prints the following and name it `MyInfo.py`:

      Hi, my name is `yourFirstname yourLastname`

      I am in my third year of `Electrical/Computer Engineering`

Copyright 2026
No Unauthorized Duplication Permitted

My student number is `XXXXXXXXX`

My MacID is `XXXXX`

10. Run both python programs and check that they are functioning correctly.
11. Take a screenshot of your terminal and save it to your computer as Step 10.

12. Rename `yourMacID_folderA` to `yourMacID_textfiles`.
13. Move all the text files from the `root` directory to `yourMacID_textfiles` using wildcards.
14. Rename the file `calculation.py` to `calculation_2#s.py`
15. Rename `yourMacID_folderB` to `yourMacID_pythonfiles`.
16. Move all the python files to `yourMacID_pythonfiles` using wildcards
17. Take a screenshot of your terminal and save it to your computer as Steps 12-16.

18. Create the following empty files in the directory `yourMacID_backup`:

    `Firstname_Lastname`
    `macID_FileX`
    `macID_FileY`
19. Copy the content of `macID_FileY` to the file `Firstname_Lastname`
20. Copy `Firstname_Lastname` and `macID_FileX` to your `home_yourMacID` directory.
21. Delete `yourMacID_backup` directory.
22. Take a screenshot of your terminal and save it to your computer as Steps 18-21.

23. View the `MyCalendar.txt` text file in calendar order.
24. View the `MyFamily.txt` text file in reverse alphabetical order and then send the contents of that file in alphabetical order to a new file called `MyFamily_alphaOrder.txt` in the same directory (Hint: use the > command as instructed in the videos)
25. Delete the `MyFamily.txt` file.
26. View the `MyDailySteps.txt` in numerical order.
27. Take a screenshot of your terminal and save it to your computer as Steps 23-26.

28. Navigate back to your root directory.
29. Find all the text files and make sure they are all in the `yourMacID_textfiles` directory.

Copyright 2026
No Unauthorized Duplication Permitted

30. Find all the python files and make sure they are all in the `yourMacID_pythonfiles` directory.
31. Search for the following words and type in your submission how many times they have been repeated: (i) Sandra, (ii) my, (iii) evening.
32. Take a screenshot of your terminal and save it to your computer as Steps 28-31.